RELATÓRIO TEÓRICO

PEDIDO-VELOZ — CLOUD DEVOPS E MICROSSERVIÇOS

Atividade acadêmica: Cloud DevOps: Orchestrating Containers and Micro Services

Este relatório apresenta os conceitos relacionados à arquitetura de microsserviços, conteinerização, orquestração, integração e entrega contínuas, observabilidade e infraestrutura como código, considerando o contexto do projeto Pedido-Veloz.

Repositório: https://github.com/Trixbeadev/Pedido-Veloz

Vídeo de apresentação: https://youtu.be/DndaMGAUI_I

1. Introdução

A construção e a operação de aplicações modernas envolvem não apenas o desenvolvimento das funcionalidades, mas também a organização dos serviços, a automação da execução, a observabilidade e a capacidade de evoluir o ambiente com segurança. O projeto Pedido-Veloz utiliza como contexto um sistema de gerenciamento de pedidos para estudar esses aspectos.

O objetivo deste relatório é descrever os principais conceitos que fundamentam as tecnologias relacionadas ao projeto. O texto apresenta uma visão conceitual e não substitui a verificação das configurações concretas existentes nos arquivos da aplicação.

2. Arquitetura de microsserviços

Microsserviços são uma abordagem arquitetural em que uma aplicação é dividida em serviços menores, organizados em torno de responsabilidades específicas. Em um sistema de pedidos, essas responsabilidades podem incluir pedidos, pagamentos e estoque. A separação favorece a organização do código e permite que cada serviço seja desenvolvido e mantido com maior independência.

A comunicação entre serviços precisa ser definida por contratos claros, como endpoints HTTP e formatos de dados. Também é necessário considerar falhas de comunicação, validação das entradas, consistência dos dados e tratamento de erros. A divisão em microsserviços, por si só, não garante escalabilidade ou disponibilidade: esses resultados dependem do desenho e da operação do sistema.

3. Conteinerização com Docker

Docker permite empacotar uma aplicação e suas dependências em uma imagem que pode ser executada como contêiner. Essa abordagem ajuda a reduzir diferenças entre ambientes de desenvolvimento e execução. Cada serviço pode ter sua própria imagem e configuração, de acordo com as necessidades da aplicação.

O Docker Compose facilita a definição e a execução conjunta de múltiplos contêineres por meio de um arquivo de configuração. Nele podem ser declarados serviços, redes, volumes, variáveis de ambiente e mapeamentos de portas. O comando de inicialização depende dos nomes e das opções realmente definidos no arquivo docker-compose.yml.

4. Persistência de dados com PostgreSQL

PostgreSQL é um sistema gerenciador de banco de dados relacional. Ele permite armazenar informações em tabelas relacionadas, aplicar restrições e realizar consultas por SQL. Em uma aplicação de pedidos, um banco relacional pode guardar pedidos, itens, pagamentos e informações de estoque, conforme o modelo de dados implementado.

A configuração de conexão, o esquema do banco, as credenciais e a persistência em volumes devem ser documentados e protegidos. Dados sensíveis não devem ser armazenados diretamente em arquivos públicos do repositório.

5. Orquestração com Kubernetes

Kubernetes é uma plataforma de orquestração de contêineres. Entre seus recursos estão Pods, Deployments, Services, ConfigMaps, Secrets e mecanismos de verificação de saúde. Esses recursos ajudam a declarar como uma aplicação deve ser executada e a manter o número desejado de réplicas.

Manifests YAML descrevem os recursos que o cluster deve criar. Para executar os arquivos, é necessário um cluster acessível e uma configuração válida do kubectl. A existência de manifests no repositório não significa, por si só, que a aplicação já esteja implantada em um cluster.

6. Integração e entrega contínuas (CI/CD)

CI/CD reúne práticas de automação para integrar alterações e preparar a entrega de software. A integração contínua pode executar verificações, testes e construção de imagens após mudanças no código. A entrega ou implantação contínua pode automatizar etapas adicionais, de acordo com as permissões e os critérios de aprovação definidos.

No GitHub Actions, os workflows são definidos em arquivos YAML normalmente armazenados em .github/workflows/. A abrangência do pipeline deve ser descrita com base nas etapas que esses arquivos realmente executam.

7. Observabilidade: métricas, logs e traces

Observabilidade é a capacidade de compreender o estado interno de um sistema a partir dos dados que ele produz. Três sinais comuns são métricas, logs e traces. As métricas ajudam a observar tendências e indicadores; os logs registram eventos; e os traces mostram o percurso de uma requisição entre componentes instrumentados.

Prometheus é utilizado para coletar e consultar métricas. Jaeger permite explorar rastreamentos distribuídos. OpenTelemetry oferece padrões e bibliotecas para gerar e encaminhar dados de telemetria, podendo atuar em conjunto com um Collector. A utilidade dessas ferramentas depende da instrumentação, das fontes de dados e das configurações de coleta.

8. Infraestrutura como código com Terraform

Infraestrutura como código (IaC) consiste em descrever recursos de infraestrutura por meio de arquivos versionáveis. Terraform permite declarar recursos e gerenciar alterações de forma reproduzível, de acordo com os provedores configurados. Arquivos de variáveis e saídas podem separar parâmetros de configuração dos recursos declarados.

Antes de aplicar configurações de infraestrutura, é importante revisar o plano de mudanças, proteger credenciais e evitar publicar estados ou segredos sensíveis. A presença de arquivos Terraform demonstra a organização dessa parte do projeto, mas os recursos efetivamente provisionados dependem da configuração e da execução.

9. Segurança e boas práticas

Algumas práticas importantes para projetos conteinerizados incluem manter segredos fora do controle de versão, usar arquivos de exemplo sem credenciais reais, revisar permissões, atualizar dependências e validar os arquivos de configuração antes da execução. Também é recomendável registrar instruções de instalação, comandos úteis e limitações conhecidas.

10. Considerações finais

O contexto do Pedido-Veloz permite relacionar desenvolvimento de microsserviços com práticas de operação, automação e monitoramento. Docker Compose pode apoiar a execução local; Kubernetes aborda a orquestração; CI/CD automatiza verificações e entregas; e as ferramentas de observabilidade ajudam a analisar o comportamento do sistema. Terraform introduz o conceito de infraestrutura declarativa.

A conclusão sobre o funcionamento de cada recurso deve ser sustentada por evidências, como execução dos serviços, testes, logs, métricas e resultados do pipeline. Este relatório teórico apresenta os fundamentos necessários para interpretar essas etapas.

Referências de consulta

- Docker Documentation — https://docs.docker.com/
- Docker Compose — https://docs.docker.com/compose/
- Kubernetes Documentation — https://kubernetes.io/docs/
- GitHub Actions Documentation — https://docs.github.com/actions
- Prometheus Documentation — https://prometheus.io/docs/
- Jaeger Documentation — https://www.jaegertracing.io/docs/
- OpenTelemetry Documentation — https://opentelemetry.io/docs/
- Terraform Documentation — https://developer.hashicorp.com/terraform/docs
- PostgreSQL Documentation — https://www.postgresql.org/docs/

Observação: os endereços acima são referências oficiais para consulta dos conceitos. A descrição de funcionalidades específicas do Pedido-Veloz deve ser conferida nos arquivos do projeto e nos resultados de execução.